Distributed Performance Computing: The Steady State Heat Equation

Q1:

First Approach:

Using OpenMPI, you can distribute and split the grids into separate units of work, using geometric decomposition.

- Taking a 12x12 grid, you can calculate such a grid in parallel by decomposing the grid into separate units of work;
- Assigning them to a thread or process.
- Have the process update one or more of these cells.
- Through message parsing, allow each unit to access different units, or domains, by copying, or 'ghosting' the neighbouring column, or row, which holds the updated calculated value from the neighbouring unit.
- Each process calculates the temperature at any given point but will calculate the values at the borders between chunks to either the north, east, south or west through message parsing.
- The process can be done using OpenMPI.

Second Approach:

Using OpenMP, we can use a shared memory approach, and use techniques such as Work Sharing or Single Program, Multiple Data (SPMD) and implement a cyclic distribution of the iteration, or, more commonly known, a round-robin distribution.

- Taking a 12x12 grid, you can define a number of threads and define a parallel block
- You can distribute separate iterations to threads, by sharing certain variables, and making other variables private to each thread.
- You call method in OpenMP to get the number of threads, assign that to a global variable out of scope, as well as a variable to get a specific thread ID number using OpenMP calls. Allocating information outside the parallel block will sit on the heap - it is available to all the threads. You work out the starting ID and ending ID of a thread by making OpenMP calls. To eliminate the amount of work this involves, you can call #pragma omp parallel for collapse(2) to do the heavy lifting for you before entering any number of loops.
- Threads execute the loop based on the starting and ending values of the thread ID.
- You can define critical, barrier or lock sections where necessary to improve performance.
- Program starts at one master thread, then it becomes split at a certain point into a team of threads. This concurrency is mapped by the programmer. When this is finished, they join back up into a thread again. These can be nested (thread 0 runs on the team too).

Choosing An Approach:

My choice of method was to implement a Work Sharing strategy with OpenMP. Work sharing in OpenMP is using a subset of directives which handle a lot of the lower level API pthread environment initialisations. It follows the concept of distributing

When OpenMP encounters the parallel for directive, threads get created and the iterations of the loop are divided among them. At the end of the parallel region, threads are suspended, waiting for the next parallel section.

- OpenMP create and suspend functionality consumes some overhead and may be unnecessary when two loops are adjacent. However, the overhead can be avoided by entering a parallel section a limited number of times, and dividing the work within the parallel section using suitable directives.
- OpenMP is more portable than OpenMPI, especially when computing a 'ghosting' technique.

- 'In irregular multi-physics applications which do not adhere solely to the SPMD mode of computation, as encountered in tightly coupled fluid-particulate systems, the flexibility of OpenMP can have a big performance advantage over MPI.(Amritkara, Taftia, Rui Liub, & Chapman, 2012).
- The program language is simple - it does not need to deal with message parsing like MPI. This makes it a suitable language for a junior developer in parallel programming, like myself.
- Incremental, and non-fatal. Directives can begin simply, and can be expanded when working.
- Not all serial statements need to be parallised. Complex calculations can work in serial too, being placed in between parallel blocks.
- A paper on Ghost Cell Pattern, by Fredrik Berg Kjolstad et al., they mention the trade off between distributed memory vs message parsing when calculating a grid cell like this - '(In Distributed Memory) less frequent communication by redundantly storing and updating cells that belong to a different process locally. This leads to some redundant computation, but may reduce time to completion if the overhead of send operations is high. However, the cost of performing the extra computation must be evaluated against the overhead cost of sending messages. There is a trade off between the two. (Kjolstad & Snir, p. 2).
- Portable multithreading in different languages include C/C++, Fortran and now, Java.
- Although not implemented, it can be used on various accelerators such as GPGPU.

Q2:
<u>Work-sharing with OpenMP in C/C++</u>

Before beginning to implement OpenMP with the serial program I had implemented with C/C++, I wanted to understand the meaning of the directives more conceptually in order to know how exactly the program could be parallelised. In distributed memory architecture, any computer composed of multiple processing elements that share an address space comprise of fall under two distinct classes:

- Symmetric Multiprocessor (SMP) - they assume that every processing element has an equal footing. It assumes that there are no special block of memory - that all memory is equally accessible at an equal cost from anywhere in the machine.
- Non-Uniform memory architecture (NUMA). All memory is equal but some memory is more equal than others. Different memory regions have different access costs - memory is segmented into 'near' and 'far' memory. Some addresses are closer than others - hence it is non-uniform. Most machines nowadays work in this form.

Choosing vectors over arrays:

There seems to be a lot of debate over which data structure is more sensible, but for ease of implementation for a less procedural program like mine, I found it easier pass the vector by reference instead of pointing to a set of pointers. In addition, information about the size and the data stored could easily be kept outside of scope too. This allowed me to brake up with program into functional blocks now stored in a header file. I knew from the beginning that as long as no methods like push_back() were used, OpenMP supported vector implementation.

Program Structure:

The program is divided up into separate functions, which is headed in with .h file. An outline of the program of some of the program's procedure is as follows:

I. Generate:

    (i) Temp, R,G and B vectors are resized in which its parameters point to the row and column values passed by reference into the function;
    (ii) All values are set to 0.

II. Initialisation:
    I. Default boundary values set, and a mean temperature for the interior cells is calculated to give an approximate starting temperature.

III. Serial Calculation:
    I. Temperature is calculated over an iterative process, the breaking point - the accumulative temperates at the current iteration and the accumulative differences at the last iteration is calculated at every iteration via a storage of temporary variables. If they equate to a difference that is equal or less than the 'tolerance' specified by the user, the while loop will break. Algorithm is measured by wall clock time at beginning and end.

IV. Parallel Calculation:
    I. The same process occurs as Serial Calculation, but sections of the algorithm are split up in a divide and conquer approach. Algorithm is measured by wall clock time at beginning and end.

V. Calculate RGB:
    I. Short Rainbow colour algorithm maps the temperature values to a switch case - colour distribution matches the allocation of colours - cold to hot.

VI. Write PPM:
    I. RGB values are written to a P3 file format.

VII. Calculate Standard Deviation between both processors.

Serial Calculation:

Once boundary values and n mean interior temperature were set, I built an algorithm to go through every interior cell in turn, and calculate a new value every time. Calculations were nested in 3 loops - do{}while ,to determine a new grid calculation, and two four loops which iterate over the 2D matrix. I had to store every temperature in a grid in an accumulative total on every grid, and compare it with the accumulative total of the last. This was the end condition of the iteration. Eventually, the difference between the last grid and the current would be the same or less than the tolerance, and the iteration would stop.

Using clock() and omp_get_wtime() to time the algorithm, and tested it with different initial interior temperature values, grid sizes, and tolerances. This information was written to a text file.

Parallel Calculation:

When writing a function to calculate heat distribution on four processors, I attempted to replicate the same calculation as in the serial process so that it could be a fair comparison. The same tolerance and grid size was used for both processes. Using clock() and omp_get_wtime() to time the algorithm, and tested it with different initial interior temperature values, grid sizes, and tolerances. This information was also written to a text file.

<u>the [#pragma omg parallel] directive</u>

The official MSDN Microsoft documentation for this directive states the #pragma omp parallel 'Defines a parallel region, which is code that will be executed by multiple threads in parallel.'

This directive requests a number of threads to be set up. The programmer can, but is not obligated to, request any number of threads. Defined thread amounts however, are not guaranteed, as this could enable the programmer to request an unreasonable number. The #pragma directive lets the system deal with lower level

API and calls 1-n* of Pthreads. The system then takes the body of the structured parallel block, packages it into a function (a thunk) and assigns a thread. When the pthread environment is set up, it launches a copy of that packaged function. The master thread which sets this up also launches itself at the last moment. When the master thread launches a team of threads, it will itself usually become thread 0 in that team. In this example, when the client makes the call:

```
#pragma omp parallel num_threads(4)
{
    foobar();

}
```

The system translates this, using pthreads at a lower-level environment, to:

```
void thunk ()
{
        foobar();
}
pthread_t tid[4];
for(int i = 1; i < 4; ++i)
{
     pthread_create (
         &tid[i],0,thunk,0);
         thunk();
}

for(int i=1l i<4;++i)
   pthread_join(tid[i]);
```

In my program, this directive was used twice within inside the scope of my while loop. That means on every iteration to calculate new temperatures for a grid, a set of threads were created. This is expensive on performance. Compared to the serial program, you will notice that the algorithm was cut in half, and that two parallel block teams of threads were created.

How does operating system manage the units of execution in a shared memory environment? The program can be divided into a region of heap, a data region, a text where the program comes from, and a stack. The way a modern operating system manages this process is that it decomposes this into threads by fragmenting the stack each thread has a chunk of the stack maintaining its own program pointer and its own register set. They share the text, data and the heap. A block of threads that share a context, but they switch between the context very cheaply. Threads will spend a short time doing something, an interleave with each other.

When the program's outcome changes as the threads are scheduled differently. To control race condition, I use synchronisation - it is however, expensive. I avoided using this where possible.

Extending the [#pragma omp parallel] directive

In my program, I had to note that there were shared variables which needed to be accessed across all threads, and private variables, which needed to be private to each thread.

The [#pragma omp for] directive

When implementing loops in a Single Program, Multiple Data (SPMD) algorithm in parallelisation, cyclic distributions of a loop, or round-robin approaches will enable a type of work-sharing to occur. Doing this usually involves the programmer implement a correct iteration clause for the calculation - this can be tricky. The crux of this problem is splitting up loop iterations between threads - the more common for scientific computing (evidence) is the cyclic distribution of the loop iterations. Start at the loop index equal to the ID, the limit the same, and increment by the number of threads (i=i+nthreads). This is also known as a round-robin distribution. In this more 'manual' implementation from my program, this:

```
#pragma omp parallel
{
        int id, r,c, Nthrds, start, end;
        id = omp_get_thread_num();
        Nthrds = omp_get_num_threads();
        start = id * N/Nthrds;
        end = (id+1) * N/Nthrds;
        if(id==Nthrds-1)end = N;
        for(r=start; r<end;r++)
        {
                for(c=start; c< end; c++)
                {
                        vec[r][c] = ((0.25)* ( (vec[r - 1][c]) + (vec[r][c + 1]) + (vec[r + 1][c]) +
(vec[r][c - 1]) ) );
                }
        }
}
```

Did not work, and did not give me the results I wanted. (I could also not work out how to nest this effectively). By using the new #pragma omp for clause, I translated it into this:

```
#pragma omp parallel shared(vec) private(r,c)
  {
    #pragma omp for
    for (r = 1; r < row - 1; r++)
    {
      for (c = 1; c < col - 1; c++)
      {
        vec[r][c] = ((0.25)* ( (vec[r - 1][c]) + (vec[r][c + 1]) + (vec[r + 1][c]) +
(vec[r][c - 1]) ) );
      }
    }
  }
```

Instead, this is replaced by the [#pragma omp for] clause. The directive essentially does all of the code above at a lower level. The down side to this is that the programmer looses control. However, depending on the magnitude of the program being developed, this is not necessarily a bad thing, and most programmers do not want to deal with the extra work, especially if their program is already complex.

The [#pragma omp for reduction] directive

Because a variable is both private to a thread but is also the subject of a reduction operation at the end of the parallel region, I used the reduction directive in the for loop.

```
#pragma omp parallel shared(diff, currentTotal,previousTotal) private (r,c)
  {
    #pragma omp for reduction (+:currentTotal)
    for (r = 1; r < row - 1; r++)
    {
      for (c = 1; c < col - 1; c++)
      {
        currentTotal+=vec[r][c];
      }
    }
  }
```

Impact on Performance

The impact on performance was measured by varying tolerance at 10,1,0.1,0.01 for a 500x500 grid on one and four processors. The results after running this program with multiple tolerances on a grid resources engine (YARCC) show that there is a wall clock speed increase. This is largely due to the fact that the program is running on multiple processors.

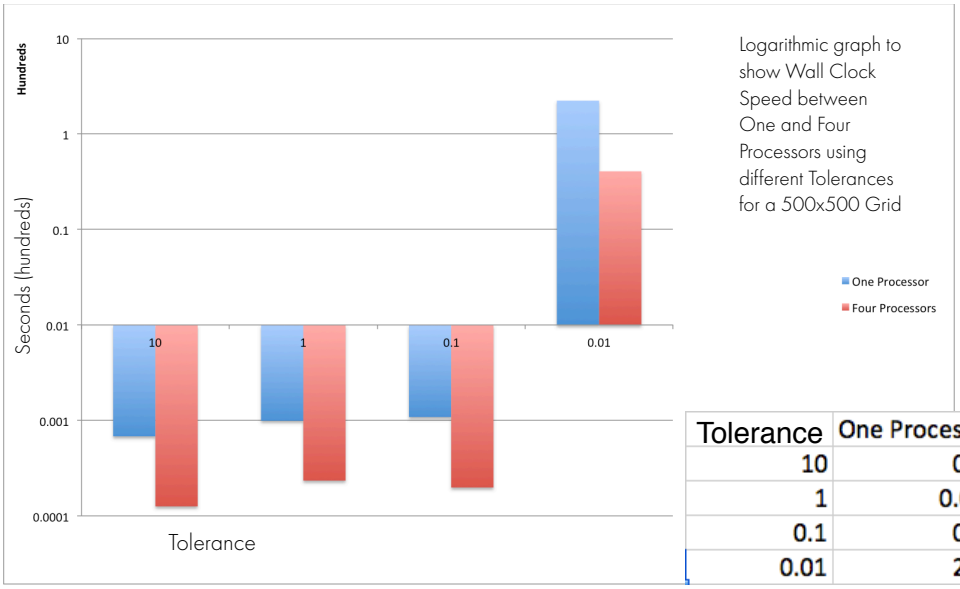


Fig.1: Graph to show Wall Clock speed.

Tolerance	One Processor	Four Processors
10	0.068229	0.0125909
1	0.0992799	0.023437
0.1	0.108562	0.019845
0.01	223.5435	40.6519

Depending on the processor, CPU time was only fractionally faster on four processors, but it was enough to see a performance increase on four processors. The following graph illustrates the results of CPU ticks for a 500x500 grid at 10,1,0.1 and 0.01 tolerance on one and four processors:

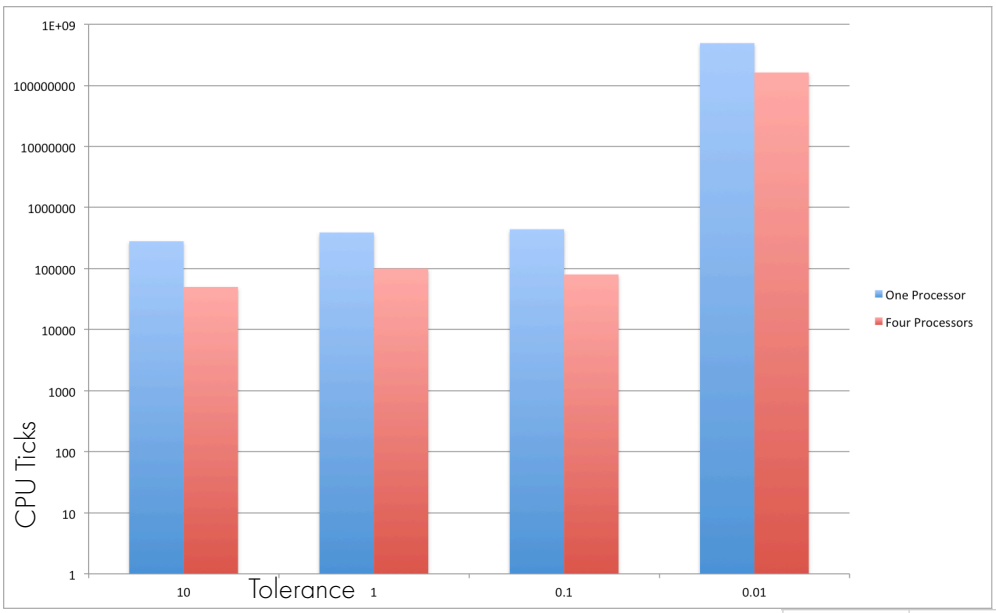


Fig 2: Graph to show CPU Ticks at different tolerances

Tolerance	One Processor	Four Processors
10	280000	50000
1	390000	100000
0.1	440000	80000
0.01	492570000	1.63E+08

The anomaly was when I ran a 1000x400 grid at a 0.01 tolerance, and found that the CPU time was greater on four processors than on one processor. See results below:

```
I
*************************
hdFourPro(): Four Processors:
Grid_SPEC: (WxH) 1000x400
Tolerance Level: 0.01
Final accumulative difference between grids: 0.00999982
Iteration Count: 245309
CPU Time(Ticks): 4.87e+08
CPU Time(Seconds): 487s
OMP Wall Time(Seconds): 121.897s
OMP Timer Precision(Seconds): 0
********************************
FINAL GRID:
************************************
```

```
*************************
hdOnePro(): One Processor:
Grid_SPEC: (WxH) 1000x400
Tolerance Level: 0.01
Final accumulative difference between grids: 0.00999995
Iteration Count: 245299
CPU Time(Ticks): 4.7566e+08
CPU Time(Seconds): 475.66s
OMP Wall Time(Seconds): 474.771s
OMP Timer Precision(Seconds): 0
********************************
FINAL GRID:
************************************
```

This shows that when running larger grid sizes of anything above 1000, the serial program will run more efficiently.

Reasons for lack of CPU speed:

- The parallel block was nested inside the first while loop, the result being that threads were being spawned on every iteration. This is performance costly. Using the collapse directive meant that I would be able to distribute inner portions of any loops that are nested, as opposed to just the first loop. However, when attempting to parallelise using this approach:

```
#pragma omp parallel for collapse(3) shared() private()
{
        while(...)
                for()
                        for() //..//
}
```

meant that the threads would individually go and calculate an entire grid. Calculations did not match what was expected, and my program continued to loop indefinitely. This leads me on to the next problem with the program - OpenMP has limited/reliable means for error handling. It has no way of informing the client that an error has occurred, or that the parallelisation is incorrect. Often I would get segmentation faults, but would find it difficult to debug this effectively. In addition, whilst work sharing apparently eliminates false sharing, this is difficult to prove and locate.

- This process could have been done more procedurally, in C, and could have been run instead on a GPU instead of a CPU. Graphics processing like this would outperform processing performance when making small calculations like this. GPUs can have thousands of lower-powered cores which are better suited for running graphical outputs. I believe this would have improved the performance drastically.
- I found that the OpenMP language had a risk of introducing difficult to debug synchronisation bugs and race conditions.

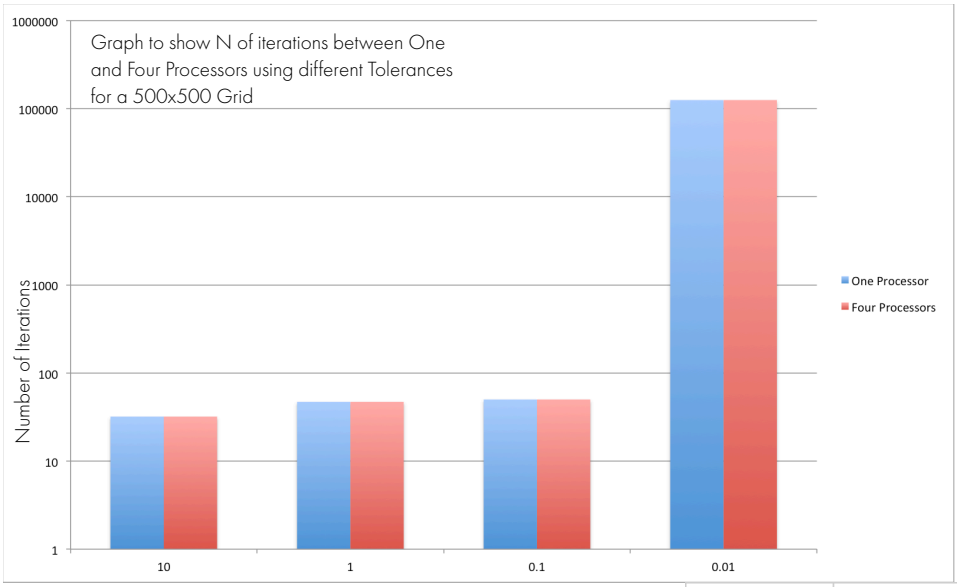


Fig.3: Graph to show no. of Iterations

The number of iterations is exactly the same on both processors.

Tolerance	One Processor	Four Processors
10	32	32
1	47	47
0.1	50	50
0.01	125338	125338

Graphical Representation:

Temperatures were written to an RGB array with a colour conversion algorithm known as 'Short Rainbow'. This enables heat to be represented as red, through to orange, yellow and then finally blue. This gradient in colour is useful in visualising calculated heat distribution on a plate.

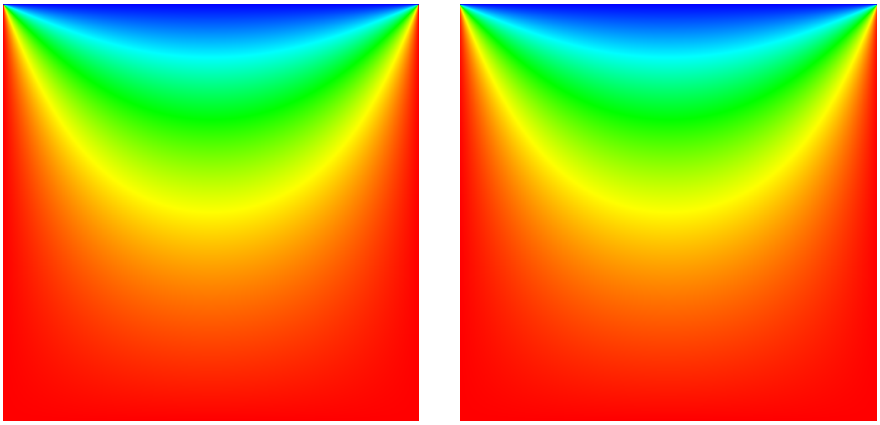


Fig.4, 5: Short Rainbow gradient .ppm image - of the final calculated temperature on a 500x500 with 0.01 tolerance on one and four processors. As you can see, there is little difference.

I also experimented with the size of the grid, producing a 400x1000 grid with 0.01 tolerance. Again, both images were identical:

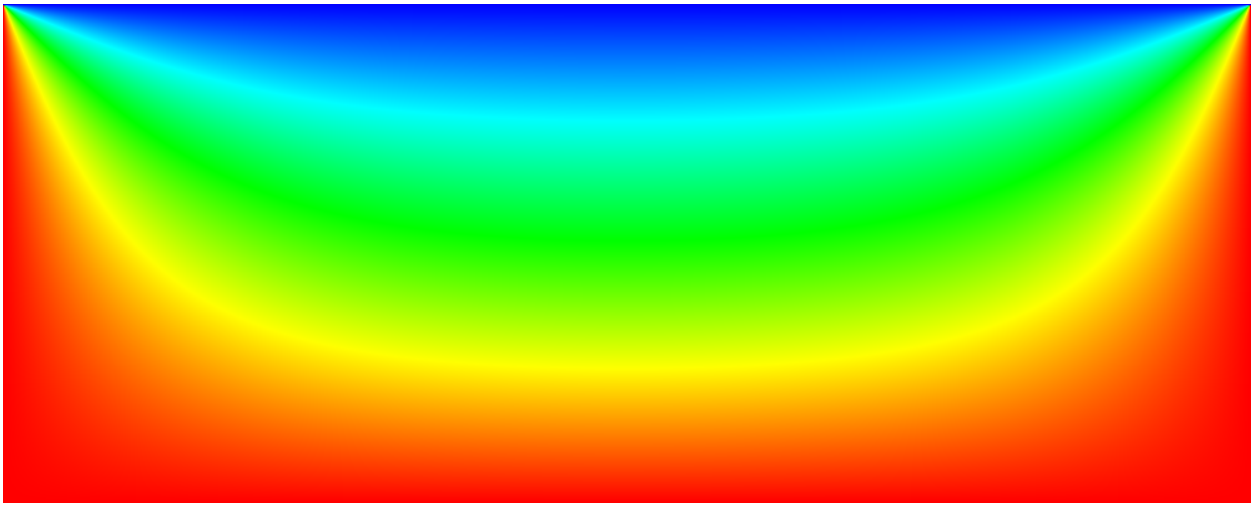


Fig 6: a 400x1000 grid with a 0.01 tolerance

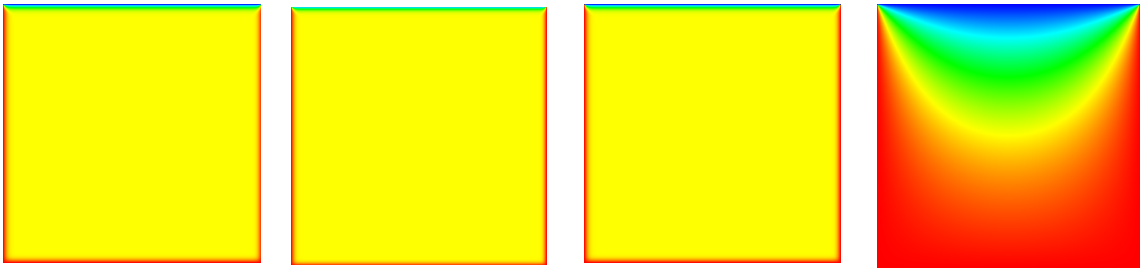


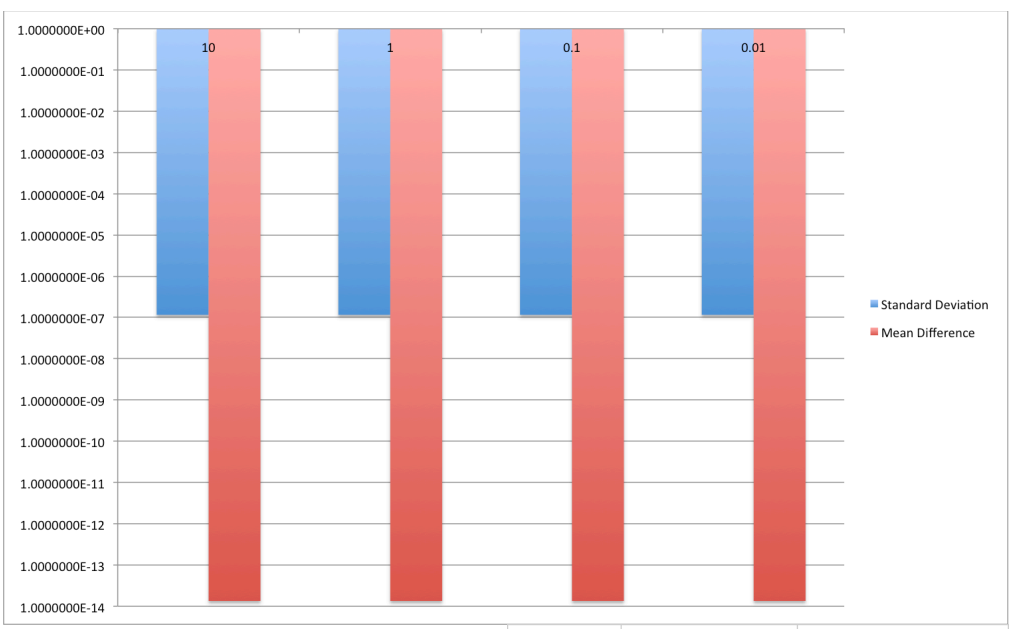
Fig 7,8,9,10: A 500x500 Grid with Tolerances (from Left to right) 10,1,0.1,0.01. We can conclude that the majority of iterations of temperature calculations are done between 0.1 and 0.01.

Calculating the standard deviation:

Here is pass in information about both vectors, iterate through one vector's size (whichever doesn't matter - they are both the same size), and subtract one from the other to find the difference, square this difference, and add these differences into an accumulative total. When outside the grid's iteration, I divide this accumulation by the grid size minus boundary values and return this to the main method. I find the standard deviation by square rooting it in the main method both the mean and standard deviation to a text file.

```
double deviation(std::vector<std::vector<double> > &vec,
      std::vector<std::vector<double> > &vec1)
      {
        double difference;
        double addedDifferences=0;
        for (int j = 1; j < vec.size()-1; j++)
        {
          for (int i = 1; i < vec[j].size()-1; i++)
          {
            if(vec[j][i] > vec1[j][i])
            {
              difference = std::pow(2.0,(vec[j][i] - vec1[j][i]));
            }
            else
            {
              difference = std::pow(2.0,(vec1[j][i]-vec[j][i]));
            }

            addedDifferences+=difference;
          }
        }
        double varianceDifferences = addedDifferences/(vec.size()-2*vec[0].size()-2);
        return varianceDifferences;
      }
```



Tolerance	Standard Deviation	Mean Difference
10	1.1595700E-07	1.34E-14
1	1.1595700E-07	1.34E-14
0.1	1.1595700E-07	1.34E-14
0.01	1.1595700E-07	1.34E-14

Fig 11: Logarithmic graph proving an equilibrium of mean difference or standard deviation comparing one and four processors and differing tolerances.

Final Thoughts on the program itself:

- The program was written in C/C++ and compiled faster than it would had it been written in Java. It does not have to run on higher level virtual machine;
- Choosing to use vectors meant I could pass information about their size, allowing me to break up the program into separate functions;
- In the algorithm design of temperature calculation, the nesting was 3-tier, and it calculations were, on the whole, limited to additions and subtractions. This is light on the ALU and CPU;
- Variables stored on the stack were kept to a minimum;

Bibliography

York, U. o. (n.d.). *YARCC*. Retrieved March 01, 2017, from York Wiki Service: https://wiki.york.ac.uk/display/RHPC/YARCC+-+York+Advanced+Research+Computing+Cluster

Amritkara, A., Taftia, D., Rui Liub, R. K., & Chapman, B. (2012, September). OpenMP parallelism for fluid and fluid-particulate systems. *Parallel Computing* , 501-517.

Hager, G., & Wellein, G. (2010). *Introduction to High Performance Computing for Scientists and Engineers*. Chapman & Hall/CRC Computational Science.

Kjolstad, F. B., & Snir, M. *Ghost Cell Pattern*. University of Illinois. Urbana-Champaign: University of Illinois.

Mattson, T. (n.d.). An Introduction to OpenMP.

Randal, E., & O'Hallaron, D. (2015). *Computer Systems - A Programmer's Perspective*. Pearson.